# Parallel Prefix Scan for State Space Models

Peter Wang, Aryan Misra, Ernest Guo, Benjamin Lindstrom
Department of Electrical and Computer Engineering
University of Southern California
Los Angeles, CA 90089

***Abstract*—State Space Models (SSMs) such as Mamba require an efficient parallel prefix scan over a custom associative operator at inference time. We implement and benchmark three GPU scan algorithms — Hillis-Steele, Blelloch up-sweep/down-sweep, and warp-shuffle — each extended to arbitrary sequence lengths via a three-phase chunked hierarchical wrapper, on an NVIDIA A100 SXM4-40GB across sequence lengths $L \in \{1024, 2048, \ldots, 131072\}$ and hidden state dimensions $D \in \{1, 16, 64, 256, 512\}$. A key architectural choice is a dimension-as-grid design that decouples shared memory cost from $D$, pinning the chunk size at the hardware maximum of 1024 regardless of hidden dimension. The warp-shuffle variant achieves up to $44\times$ speedup over a sequential CPU baseline and peak throughput of $\sim$470 GB/s at $D = 512$, $L = 131072$. Contrary to expectation, Hillis-Steele outperforms Blelloch at $D \geq 64$.**

***Index Terms*—parallel prefix scan, state space models, CUDA, warp shuffle, Mamba, GPU**

## I. Introduction and Motivation

Sequence modeling is a fundamental machine learning paradigm for interpreting ordered data. It underlies natural language processing, speech recognition, genomic sequence analysis, and time-series forecasting. The *transformer* architecture currently dominates this space through its self-attention mechanism, which allows every element in a sequence to attend to every other. This enables substantial data parallelism but scales quadratically with sequence length $L$, incurring $O(L^2)$ time and memory costs that become prohibitive for long contexts.

*State Space Models* (SSMs) offer a linear-complexity alternative, mapping input sequences to outputs through a hidden state updated at each timestep. The update is a recurrence, with each state depending on the previous one, making the naïve evaluation sequential and forfeiting the GPU parallelism that makes transformers practical. The key to recovering parallelism is that the SSM recurrence defines a *binary associative operator*, enabling a *parallel prefix scan* to compute all hidden states simultaneously. The parallel prefix scan is therefore the core computational primitive for efficient SSM inference.

This paper systematically studies these tradeoffs, which remain underexplored despite the growing adoption of SSMs. We make the following contributions:

- We implement three parallel scan algorithms — Hillis-Steele, Blelloch, and warp-shuffle — and a chunked hierarchical wrapper that extends each to arbitrary sequence lengths, targeting the SSM prefix scan primitive and benchmarked on an NVIDIA A100 across $L \in \{1024, 2048, \ldots, 131072\}$ and $D \in \{1, 16, 64, 256, 512\}$.
- We propose a scalar, dimension-as-grid execution strategy that decouples shared memory cost from $D$, keeping chunk size pinned at the hardware maximum of 1024 regardless of hidden dimension.
- We identify a non-obvious performance inversion where Hillis-Steele outperforms Blelloch at large $D$ despite its $O(L \log L)$ work cost, driven by Blelloch's heavier synchronization overhead.
- We characterize all kernels on the roofline model, showing the SSM scan is firmly memory-bound across all tested $(L, D)$, and also analyze resulting throughput, SM utilization, and occupancy.

Our hypotheses are: **(1)** arithmetic intensity rises with both $L$ and $D$, with kernels being memory-bound at large $D$; **(2)** Hillis-Steele's $O(L \log L)$ work cost produces measurable throughput degradation relative to $O(L)$ algorithms past a crossover threshold in $L$; **(3)** warp-shuffle outperforms shared-memory algorithms at small-to-moderate $D$ where register pressure is low, but may converge toward shared-memory approaches at very large $D$.

## II. Related Work

**Hillis and Steele** [1] introduced the first practical parallel prefix scan, demonstrating that a wide class of serial algorithms could be recast as data-parallel prefix computations. Their algorithm runs in $O(\log L)$ depth but performs $O(L \log L)$ total work.

**Blelloch** [2] addressed the work issue, presenting a two-phase scan with $O(L)$ total work and $O(\log L)$ depth. The up-sweep (reduce) phase constructs a binary reduction tree; the down-sweep (distribute) phase propagates exclusive prefixes. This became the standard reference for efficient parallel prefix computation.

**Harris, Sengupta, and Owens** [3] adapted Blelloch's algorithm to GPU hardware in GPU Gems 3, Chapter 39, detailing bank conflict avoidance via conflict-free shared memory addressing and introducing the hierarchical chunking technique we use as our cross-block wrapper. Our implementation omits conflict-free addressing as the dominant bottleneck in our workload is memory bandwidth, not bank conflicts.

**Gu et al.** [4] introduced *S4*, which parallelized SSM recurrences as global convolutions in the frequency domain under a fixed, input-independent state transition matrix. While efficient, the fixed-matrix assumption limits expressiveness.

**Smith et al.** [5] introduced *S5*, the first model to apply a parallel associative scan directly to SSM recurrences using a binary operator like what we do.

**Gu and Dao** [6] introduced *Mamba*, extending SSMs with input-dependent state transitions, where $\bar{A}_t$ and $\bar{B}_t$ are computed from the current token at each timestep. This breaks the convolution trick of S4, requiring the associative scan at inference time. Mamba's production `selective_scan_cuda` kernel fuses the scan with surrounding operations and does not expose the algorithmic variant used or its performance tradeoffs, motivating our independent analysis.

While prior work establishes the complexity of each scan variant, to our knowledge, no systematic empirical comparison exists on modern GPU hardware across the joint $(L, D)$ parameter space relevant to SSM inference. Our work fills this gap.

## III. Background

### A. SSM Recurrence

A State Space Model maps an input sequence $u_0, u_1, \ldots, u_L$ (where $u_t \in \mathbb{R}^m$) to an output sequence $y_0, \ldots, y_L$ (where $y_t \in \mathbb{R}^n$) via a hidden state $x_t \in \mathbb{R}^D$. In discretized form:

$$x_t = \bar{A}_t \cdot x_{t-1} + \bar{B}_t \cdot u_t, \tag{1}$$

$$y_t = C_t \cdot x_t. \tag{2}$$

where $x_t$ is the hidden state, $u_t$ is the input, and $\bar{A}$, $\bar{B}$, and $C$ are parameter matrices. In Mamba, $\bar{A}_t$, $\bar{B}_t$, $C_t$ are input-dependent, being recomputed from the current token at each timestep.

### B. Associative Operator and Prefix Scan

Although each $x_t$ depends on $x_{t-1}$, repeated substitution gives:

$$\begin{aligned} x_t = &(\bar{A}_t \bar{A}_{t-1} \cdots \bar{A}_1)\, x_0 \\ &+ (\bar{A}_t \bar{A}_{t-1} \cdots \bar{A}_2)\, \bar{B}_1 u_1 \\ &+ (\bar{A}_t \bar{A}_{t-1} \cdots \bar{A}_3)\, \bar{B}_2 u_2 \\ &+ \cdots \\ &+ \bar{A}_t \bar{B}_{t-1} u_{t-1} \\ &+ \bar{B}_t u_t. \end{aligned}$$

or simplified:

$$x_t = \left( \prod_{i=1}^{t} \bar{A}_i \right) x_0 + \sum_{k=1}^{t} \left( \prod_{i=k+1}^{t} \bar{A}_i \right) \bar{B}_k u_k.$$

This allows us to define a binary operator $\oplus$ over pairs $(a, b)$, where $a$ represents a transition scalar and $b$ an accumulated input contribution:

$$(a_2, b_2) \oplus (a_1, b_1) \;=\; (a_2 \cdot a_1,\;\; a_2 \cdot b_1 + b_2). \tag{3}$$

One can verify $\oplus$ is associative: $(c \oplus b) \oplus a = c \oplus (b \oplus a)$. Because $\oplus$ is associative, a parallel prefix scan over the sequence $(a_1, b_1), (a_2, b_2), \ldots, (a_L, b_L)$ computes all prefix products simultaneously, yielding every hidden state $x_1$ through $x_L$ in $O(\log L)$ parallel steps rather than $O(L)$ serial steps. Table I illustrates the depth reduction.

TABLE I
Sequential vs. parallel evaluation for $L = 4$.

| **Sequential (4 steps)** | **Blelloch parallel (2 steps)** |
|---|---|
| $x_1 \leftarrow a_1 x_0 + b_1$ | Step 1: $P_{12} = (a_2, b_2) \oplus (a_1, b_1)$, $P_{34} = (a_4, b_4) \oplus (a_3, b_3)$ |
| $x_2 \leftarrow a_2 x_1 + b_2$ | Step 2: $P_{1234} = P_{34} \oplus P_{12}$ |
| $x_3 \leftarrow a_3 x_2 + b_3$ | (down-sweep distributes partial results) |
| $x_4 \leftarrow a_4 x_3 + b_4$ | |

### C. Diagonal $\bar{A}$ and Independent Dimensions

In Mamba, $\bar{A}_t$ is **diagonal**. Rather than a full $D \times D$ matrix, it is parameterized as a length-$D$ vector of scalars. The $D$-dimensional recurrence therefore decouples into $D$ independent scalar recurrences:

$$x_t[d] \;=\; a_t[d] \cdot x_{t-1}[d] \;+\; b_t[d], \quad d = 0, 1, \ldots, D-1. \tag{4}$$

Because $\bar{A}_t$ is diagonal, $x_t[d]$ depends only on $x_{t-1}[d]$, $a_t[d]$, and $b_t[d]$, with no cross-dimension terms appearing. The combine operation (3) applied to dimension $d$ is therefore independent of all other dimensions, mapping directly to grid-level parallelism in CUDA, as detailed in Section V.

## IV. Algorithm Analysis

### A. Hillis-Steele

Hillis-Steele computes an inclusive prefix scan in $\lceil \log_2 L \rceil$ parallel steps. In each step, every element at index $i$ combines with the element *offset* positions to its left:

**Algorithm 1** Hillis-Steele inclusive prefix scan

```
for offset = 1, 2, 4, ..., L/2 do
    for i = 0 to L − 1 in parallel do
        if i ≥ offset then
            out[i] ← in[i − offset] ⊕ in[i]
        else
            out[i] ← in[i]
        end if
    end for
    swap(in, out)
    __syncthreads()
end for
```

**Work complexity:** $O(L \log L)$. Each of $O(\log L)$ steps performs $O(L)$ combines.

**Span (depth):** $O(\log L)$. A parallel step per $\lceil \log_2 L \rceil$ offset.

**Shared memory:** $4 \times CHUNK_SIZE \times 4$ bytes $= 16$ KB (two arrays, each double-buffered to avoid reading and writing the same array within a step).

**Algorithm 2** Blelloch inclusive prefix scan

```
// Up-sweep (reduce): build partial-sum tree bottom-up
for stride = 2, 4, ..., L do
    for i = 0 to L/stride − 1 in parallel do
        ai ← stride · i + stride/2 − 1
        bi ← stride · i + stride − 1
        s[bi] ← s[ai] ⊕ s[bi]
    end for
    __syncthreads()
end for
total ← s[L − 1];  s[L − 1] ← identity
// Down-sweep (distribute): push exclusive prefixes top-down
for stride = L, L/2, ..., 2 do
    for i = 0 to L/stride − 1 in parallel do
        ai ← stride · i + stride/2 − 1
        bi ← stride · i + stride − 1
        t ← s[ai];  s[ai] ← s[bi]
        s[bi] ← s[bi] ⊕ t
    end for
    __syncthreads()
end for
// Inclusive correction: s[i] ← s[i] ⊕ orig[i]
```

**Algorithm 3** Warp-shuffle inclusive prefix scan

```
// Each thread loads its input into registers
va ← a[tid];  vb ← b[tid]
// va, vb: this thread's running prefix (updated each step)
// la, lb: values fetched from off lanes to the left
// Phase 1: intra-warp Hillis-Steele scan (5 steps, offsets 1–16)
// offsets stop at 16 = warp_size/2; offset 32 would exit the warp
for off = 1, 2, 4, 8, 16 do
    la, lb ← __shfl_up_sync(va, off), __shfl_up_sync(vb, off)
    if lane ≥ off then
        vb ← va · lb + vb;  va ← va · la
    end if
end for
Save warp total: wt_a[wid] ← va,  wt_b[wid] ← vb
__syncthreads()
// Phase 2: first warp scans the 32 per-warp totals
if wid = 0 then
    scan wt_a[·], wt_b[·] (same 5-step loop)
end if
__syncthreads()
// Phase 3: add preceding warp's inclusive prefix
if wid > 0 then
    (va, vb) ← (wt_a[wid−1], wt_b[wid−1]) ⊕ (va, vb)
end if
```

### B. Blelloch Up-Sweep / Down-Sweep

Blelloch achieves $O(L)$ work via two tree-traversal phases.

**Work complexity:** $O(L)$. Up-sweep and down-sweep each perform $L/2+L/4+\cdots+1 = L-1$ combines since the number of active threads halves each level, in contrast to Hillis-Steele where all $L$ threads run at every step.

**Span:** $O(\log L)$. Similar to Hillis, the tree has $\log L$ levels.

**Shared memory:** $2 \times CHUNK_SIZE \times 4 = 8$ KB, and also one in-place buffer for each of the $a$ and $b$ arrays.

### C. Warp-Shuffle

The warp-shuffle algorithm applies the **Hillis-Steele** offset-doubling pattern entirely *in registers*, replacing shared memory reads and writes with CUDA's `__shfl_up_sync` intrinsic, which exchanges register values between threads within a warp without any memory transaction. Hillis-Steele's pattern, where each thread combines with a neighbor at offset $1, 2, 4 \ldots$, maps naturally to shuffle instructions because every thread executes the same operation at every step, unlike Blelloch's tree traversal where active threads vary per level. Note in the algorithm the offsets stop at $16 = \text{warp_size}/2$; an offset of 32 would exit the warp. The algorithm proceeds in three phases as described in the pseudo-code.

**Work complexity:** $O(L)$, like Hillis-Steele.

**Span:** $O(\log L)$, like Hillis-Steele.

**Shared memory:** $2 \times 1024 \times 4 = 8$ KB for the primary buffers, plus 256 bytes for warp totals.

### D. Chunked Hierarchical Wrapper

All three algorithms above operate within a single thread block of $CHUNK_SIZE = 1024$ elements. For $L > 1024$, we apply a standard three-phase hierarchical strategy [3]:

1) **Local scan.** Each block $b$ runs an independent scan over its chunk of $CHUNK_SIZE$ elements, writing results to the output and saving the block's total in `block_totals`[$b$].
2) **Scan of totals.** One single block scans the block results `block_totals`[$0 \ldots num_chunks-1$], producing the exclusive prefix sum across blocks.
3) **Propagation.** Each block $b > 0$ adds `block_totals`[$b-1$] to every element in its output chunk, making results globally correct.

The wrapper is implemented as a CUDA template parameterized on the scan type, so `HillisSteele`, `Blelloch`, or `WarpShuffle` can be substituted. For $num_chunks \leq 1024$, Phase 2 runs as a single-block scan, accommodating up to $L = 1024^2 \approx 10^6$ tokens. For larger $L$, Phase 2 recurses into `chunked_scan_impl` on the totals array.

The hierarchical strategy achieves load balance where every block performs exactly $CHUNK_SIZE$ combines in Phase 1 regardless of its position in the sequence. Inter-block communication is limited to Phases 2 and 3, which together touch only the $num_chunks$-element block totals array. During the local scan (Phase 1), blocks operate independently with zero cross-block data movement.

## V. Implementation Details

### A. Platform

All kernels are written in CUDA C++ and compiled with `nvcc` targeting `sm_80` (Ampere). Benchmarks run on the **NVIDIA A100 SXM4-40GB** GPU. Key hardware parameters include 108 SMs, 2 TB/s HBM2e bandwidth, 40 GB HBM2e, 312 TFLOPS FP32, 96 KB configurable L1/shared memory per SM, 1024 threads maximum per block.

### B. Scalar Data Layout and the Dimension-as-Grid Design

A straightforward CUDA mapping would group all $D$ scalars into a single `Element` struct per sequence position, loading the full vector into shared memory per thread. Under this mapping, shared memory consumption per block scales as `CHUNK_SIZE` $\times 2D \times 4$ bytes, forcing `CHUNK_SIZE` to shrink as $D$ grows and directly degrading the $L$-parallelism that the hierarchical wrapper (Section IV-D) is designed to provide. Since each dimension $d$ is an independent scalar scan (Section III-C), we resolve this by organizing the data layout per-dimension. We store $a$ and $b$ as flat `float` arrays indexed by dimension $d$ and timestep $t$:

```
a_ptr[d*L + t]  // transition scalar
b_ptr[d*L + t]  // input projection
```

Each thread block processes 1024 consecutive timesteps for a single dimension $d =$ `blockIdx.y`. The grid is shaped `(ceil(L/CHUNK_SIZE), D, 1)`, so blocks tile the sequence in $x$ and cover all $D$ dimensions in $y$.

This layout has three important properties. **First**, shared memory cost is independent of $D$. Each block scans a one-dimensional sequence of scalars, requiring $4 \times 1024 \times 4 =$ 16 KB of shared memory per block regardless of $D$, so `CHUNK_SIZE` is always pinned at the hardware maximum of 1024. **Second**, register pressure is constant: each thread holds only two floats $(v_a, v_b)$ regardless of $D$. **Third**, HBM accesses are perfectly coalesced. Threads within a warp share the same d (`blockIdx.y`) and hold consecutive t values (`threadIdx.x`). Because the data is laid out as $a_ptr[d * L + t]$, adjacent threads read adjacent addresses, allowing the hardware to merge a full warp's loads into a single memory transaction.

Table II quantifies the chunk-size reduction under the struct-based alternative.

TABLE II
Max chunk $L$ per block under struct-based Element design (old implementation). Scalar design always achieves $CHUNK_SIZE = 1024$ regardless of $D$.

| $D$ | Blelloch | Hillis-Steele | Warp-Shuffle |
|---|---|---|---|
| 16 | 1024 | 512 | 1024 |
| 64 | 256 | 128 | 256 |
| 256 | 64 | 32 | 64 |
| 512 | 32 | 16 | 32 |

The scalar design also preserves the load balance described in Section IV-D across the $D$ axis: every block scans exactly 1024 scalars regardless of which $d$-lane it handles, so no block is assigned disproportionate work.

### C. Correctness Validation

Benchmarks target the scan primitive in isolation, excluding the surrounding linear projections and discretization steps of a full Mamba layer. Performance depends only on $L$, $D$ and memory access patterns, not on how the $(a, b)$ arrays were produced, so we generated synthetic inputs which would be roughly equivalent to real SSM activations for benchmarking.

A sequential CPU reference iterates the recurrence directly, and all GPU kernels compare their output against the reference at float32 tolerance $10^{-4}$. Synthetic inputs use transitions $a_t \sim \mathrm{Uniform}(0.5, 0.99)$, chosen so that long products $\prod a_t$ remain numerically stable at $L = 131072$ without underflowing to zero, and projections $b_t \sim \mathcal{N}(0, 1)$. All kernels achieve 100% correctness across all $(L, D)$ combinations.

### D. Benchmarking

Wall-clock kernel time is measured with CUDA events. Each kernel launch is bracketed by `cudaEventRecord` calls, with `cudaEventSynchronize` ensuring host-side accuracy. Memory throughput is computed as $2 \times L \times D \times 2 \times$ $sizeof(\texttt{float})/t_{\mathrm{ms}}$, accounting for one read and one write of both the $a$ and $b$ arrays. Each kernel configuration is timed with a single run. GPU kernel execution is largely deterministic for fixed grid and block dimensions so we'll leave multi-trial averaging to future work. Additional hardware metrics such as achieved warp occupancy, SM utilization, HBM bandwidth utilization, and roofline position, are collected via NVIDIA Nsight Compute on the same A100 SXM4 instance.

## VI. Results

### A. Wall-Clock Time and Speedup

Table III presents wall-clock kernel times and speedup over the sequential CPU baseline at representative $(D, L)$ combinations. At $D = 512$, $L = 131072$, warp-shuffle achieves a $\mathbf{44}\times$ speedup over sequential, reducing a 76 ms CPU computation to 1.7 ms.

Figure 1 shows speedup over sequential at $L = 131072$ across all $D$ values. Warp-shuffle speedup grows steadily with $D$, from $25.7\times$ at $D = 1$ to $44.5\times$ at $D = 256$, before plateauing at $D = 512$, consistent with the memory subsystem approaching saturation. Hillis-Steele follows a similar trend, reaching $34.5\times$ at $D = 512$. Blelloch's speedup, by contrast, remains nearly flat across all $D$ ($18\times$–$21\times$), suggesting its performance is bottlenecked by synchronization overhead rather than memory bandwidth, and thus does not benefit from the additional parallelism that larger $D$ provides.

### B. Kernel Comparison

Figure 2 presents per-kernel timing curves for $D = 512$; timing curves for $D \in \{1, 16, 64, 256\}$ follow roughly similar trends and are provided in Appendix A. Three patterns appear across $D$ values. First, warp-shuffle is the fastest kernel at

TABLE III
KERNEL TIME (MS) AND SPEEDUP OVER SEQUENTIAL CPU BASELINE (SELECTED RESULTS).

| $D$ | $L$ | Seq. (ms) | Warp-Shuf. | | Blelloch | | Hillis-Steele | |
|---|---|---|---|---|---|---|---|---|
| | | | ms | × | ms | × | ms | × |
| 16 | 1K | 0.016 | 0.015 | 1.0 | 0.018 | 0.9 | 0.016 | 1.0 |
| 16 | 8K | 0.134 | 0.021 | 6.3 | 0.031 | 4.3 | 0.027 | 5.0 |
| 16 | 32K | 0.567 | 0.031 | 18.2 | 0.051 | 11.1 | 0.039 | 14.4 |
| 16 | 128K | 2.440 | 0.061 | 40.3 | 0.128 | 19.1 | 0.086 | 28.4 |
| 64 | 1K | 0.048 | 0.022 | 2.1 | 0.027 | 1.8 | 0.025 | 1.9 |
| 64 | 8K | 0.563 | 0.031 | 18.1 | 0.051 | 11.1 | 0.040 | 14.1 |
| 64 | 32K | 2.485 | 0.062 | 39.8 | 0.130 | 19.1 | 0.085 | 29.3 |
| 64 | 128K | 9.299 | 0.235 | 39.5 | 0.529 | 17.6 | 0.348 | 26.7 |
| 256 | 1K | 0.208 | 0.028 | 7.5 | 0.039 | 5.3 | 0.031 | 6.8 |
| 256 | 8K | 2.553 | 0.066 | 38.9 | 0.143 | 17.9 | 0.097 | 26.3 |
| 256 | 32K | 9.487 | 0.236 | 40.2 | 0.528 | 18.0 | 0.347 | 27.4 |
| 256 | 128K | 37.90 | 0.874 | 43.4 | 1.996 | 19.0 | 1.254 | 30.2 |
| 512 | 1K | 0.410 | 0.031 | 13.2 | 0.049 | 8.3 | 0.038 | 10.8 |
| 512 | 8K | 4.381 | 0.121 | 36.1 | 0.251 | 17.5 | 0.170 | 25.8 |
| 512 | 32K | 18.97 | 0.449 | 42.2 | 1.037 | 18.3 | 0.668 | 28.4 |
| 512 | 128K | 75.96 | 1.715 | 44.3 | 3.576 | 21.2 | 2.226 | 34.1 |

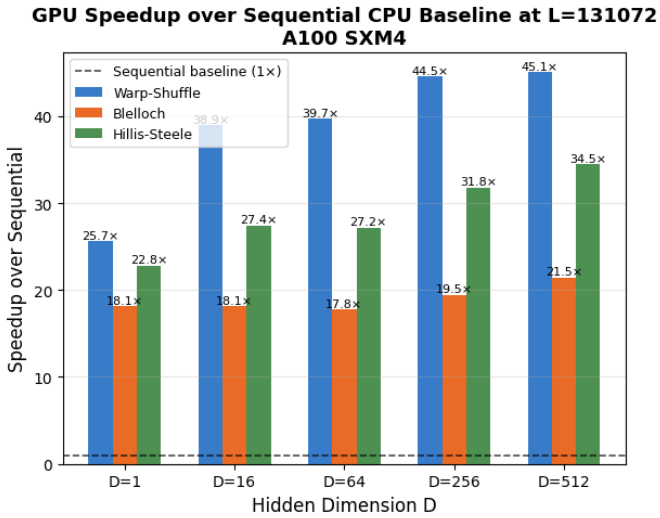


Fig. 1. Speedup over sequential CPU baseline at $L = 131072$ across hidden state dimensions $D \in \{1, 16, 64, 256, 512\}$. Blelloch speedup is nearly flat with $D$; warp-shuffle and Hillis-Steele grow with $D$ until memory bandwidth saturates.

most $(L, D)$ combinations, with its advantage over the shared-memory kernels growing as $L$ increases. Second, all three GPU kernels converge to similar times at small $L$ (1K–4K), where GPU launch overhead dominates over computation, consistent with Amdahl's Law. Third, the gap between sequential and all GPU kernels widens consistently with both $L$ and $D$, as the parallel workload grows large enough to fully utilize the GPU.

A notable result visible in Figure 2 is that **Hillis-Steele outperforms Blelloch at** $D \geq 64$ despite its higher $O(L \log L)$ work complexity. At $D = 512$, $L = 131072$, Hillis-Steele runs

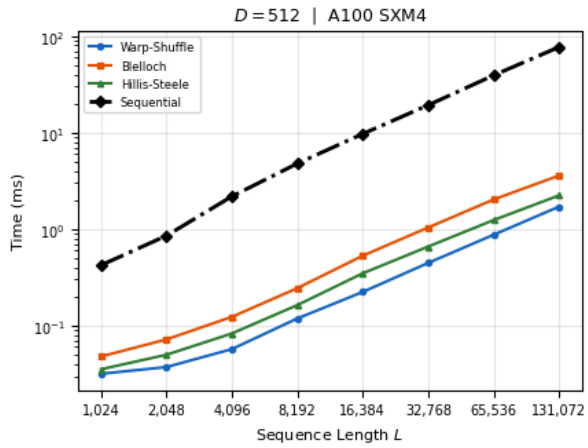


Fig. 2. Per-kernel timing at $D = 512$. Warp-shuffle achieves 44× speedup over sequential at $L = 131072$. Hillis-Steele outperforms Blelloch at all tested $L$.

in 2.2 ms versus Blelloch's 3.6 ms, a 1.6× difference in favor of the theoretically work-inefficient algorithm. This inversion is consistent across all tested $L$ at large $D$ and is analyzed in Section VII.

### C. Memory Throughput

Table IV reports memory throughput (GB/s) for all three kernels. The A100's nominal HBM2e bandwidth is ∼2000 GB/s; peak achieved throughput across all kernels is ∼470 GB/s (∼23% utilization), placing every kernel firmly in the memory-bound regime. This is consistent with the scan's arithmetic intensity of approximately 3 FLOPs/(2 × 4 bytes) = 0.375 FLOP/byte, well below the A100's ridge point.

TABLE IV
MEMORY THROUGHPUT (GB/S) AT SELECTED $(L, D)$. A100 PEAK ≈ 2000 GB/S.

| Kernel | | $D = 16$ | $D = 64$ | $D = 256$ | $D = 512$ |
|---|---|---|---|---|---|
| Warp-Shuffle | $L = 32K$ | 202 | 403 | 427 | 448 |
| | $L = 128K$ | 416 | 428 | 461 | 470 |
| Hillis-Steele | $L = 32K$ | 160 | 297 | 290 | 302 |
| | $L = 128K$ | 293 | 289 | 321 | 362 |
| Blelloch | $L = 32K$ | 123 | 193 | 191 | 194 |
| | $L = 128K$ | 196 | 190 | 202 | 225 |

Warp-shuffle throughput rises with both $L$ and $D$, plateauing near 470 GB/s. Hillis-Steele follows a similar trend, reaching 362 GB/s at $D = 512$. Blelloch, however, achieves substantially lower throughput, only 225 GB/s at $D = 512$, $L = 131072$ or roughly half that of warp-shuffle. Critically, Blelloch's throughput is also nearly flat across $D$ (190–225 GB/s at $L = 128$K), mirroring the flat speedup observed in Figure 1. This indicates that Blelloch's time is consumed by synchronization barriers rather than memory transactions: the $2\lceil \log_2 CHUNK_SIZE \rceil = 20$ `__syncthreads` calls per

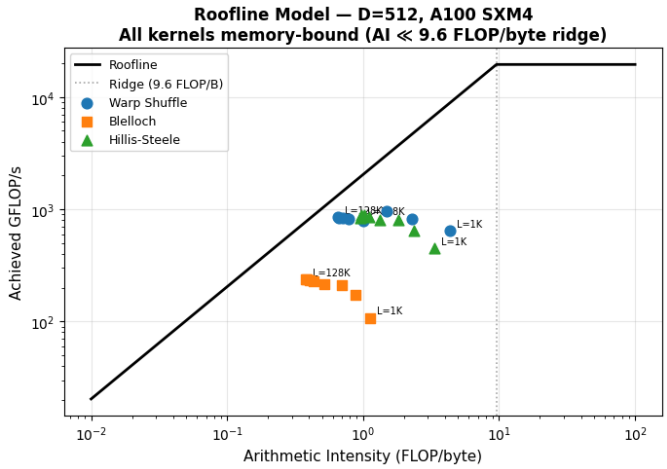


Fig. 3. Roofline model at $D = 512$. All kernels are memory-bound (well left of the 9.6 FLOP/byte ridge). Warp-shuffle and Hillis-Steele approach the memory bandwidth ceiling at large $L$; Blelloch achieves 3–4× lower GFLOP/s at the same arithmetic intensity, consistent with synchronization stalls consuming cycles without memory transactions.

block scan stall warps at each tree level, leaving the memory bus underutilized.

### D. Roofline Analysis

Figure 3 places all three kernels on the roofline model at $D = 512$. The A100's ridge point is 9.6 FLOP/byte. All kernels cluster between 0.5 and 5 FLOP/byte, well to the left of the ridge, confirming that the SSM scan is **memory-bound** across all tested $(L, D)$: performance is limited by HBM bandwidth, not arithmetic throughput. This is consistent with the scan's theoretical arithmetic intensity of 3 FLOPs/8 bytes = 0.375 FLOP/byte.

The roofline also shows that Hillis-Steele achieves 800–1000 GFLOP/s at large $L$, approaching the memory bandwidth bound, while Blelloch clusters near 200–300 GFLOP/s despite similar arithmetic intensity. This is consistent with the synchronization overhead identified in Section VI-C. Points shift *leftward* as $L$ grows (labeled in the figure), reflecting the transition from cache-resident execution at small $L$ to full HBM streaming at large $L$, where measured bytes transferred approaches the true data volume.

Roofline plots for $D \in \{1, 16, 64, 256\}$ are provided in Appendix A and show consistent memory-bound behavior across all tested dimensions.

### E. Occupancy

Figure 4 shows achieved warp occupancy averaged across all sequence lengths for each kernel and $D$ value, measured via Nsight Compute. At $D = 1$, all three kernels achieve only ∼48–50% occupancy: with a single block column in the grid's $y$-dimension, fewer blocks are available to fill SM warp slots. As $D$ increases, the 2D grid exposes $D$ independent block columns simultaneously, giving the SM scheduler a larger pool of resident warps. By $D = 256$ and $D = 512$, occupancy

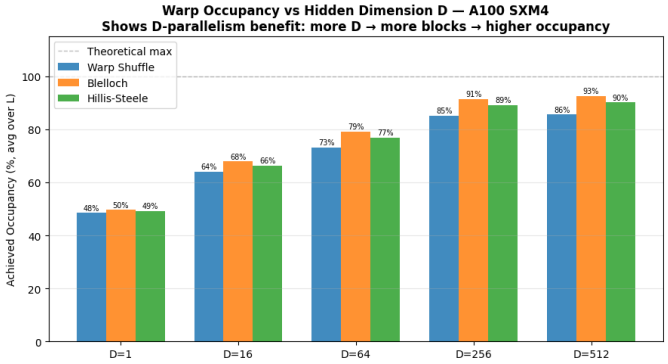


Fig. 4. Achieved warp occupancy (averaged over $L$) vs. hidden dimension $D$. Occupancy rises from ∼48% at $D = 1$ to ∼85–93% at $D = 512$, quantifying the benefit of the dimension-as-grid design. Blelloch consistently achieves the highest occupancy due to lower shared memory pressure per block; warp-shuffle is marginally lower owing to its slightly larger warp-totals buffer.

reaches 85–93%, near the ceiling imposed by shared memory allocation per block.

The $D = 1$ baseline isolates the contribution of $D$-parallelism cleanly: it has identical $L$-parallelism to $D = 512$ but none of the $D$-parallelism, and occupancy is nearly halved as a result. This directly quantifies the architectural gain from treating $D$ as a grid dimension rather than a loop variable inside each thread.

### F. Streaming Multiprocessor vs. HBM Bandwidth Utilization

Figure 5 plots SM utilization against HBM bandwidth utilization for each kernel across all $(L, D)$ combinations. The position relative to the diagonal does not directly indicate whether a kernel is compute- or memory-bound (that is established by the rootline plots in Section VI-D). Rather, this plot characterizes the balance between SM and HBM activity for each kernel.

Hillis-Steele points mostly sit *above* the diagonal at large $D$ (SM ∼50–55%, HBM ∼40–50%), indicating the SM is more active than the memory bus. Blelloch points cluster near the diagonal (SM ≈ HBM ≈ 40–42%), with both metrics lower than Hillis-Steele.

The most interpretable finding from this plot is Blelloch's behavior: its $2\lceil \log_2 L \rceil = 20$ `__syncthreads` barriers per block scan stall all warps simultaneously, suppressing both SM and HBM utilization together and leaving both resources underutilized relative to the other kernels. Additionally, in a memory-bound regime, Hillis-Steele's extra $O(L \log L)$ arithmetic work may not incur additional wall-clock cost, since the SM has idle cycles while waiting on HBM transactions that can absorb the extra computation for free.

## VII. Discussion

### A. Hypothesis Validation

**Arithmetic intensity rising with $L$ and $D$.** Partially confirmed. Achieved throughput (GB/s) rises with both $L$ and $D$ (Table IV), consistent with the working set growing large enough to saturate HBM. However, arithmetic intensity as

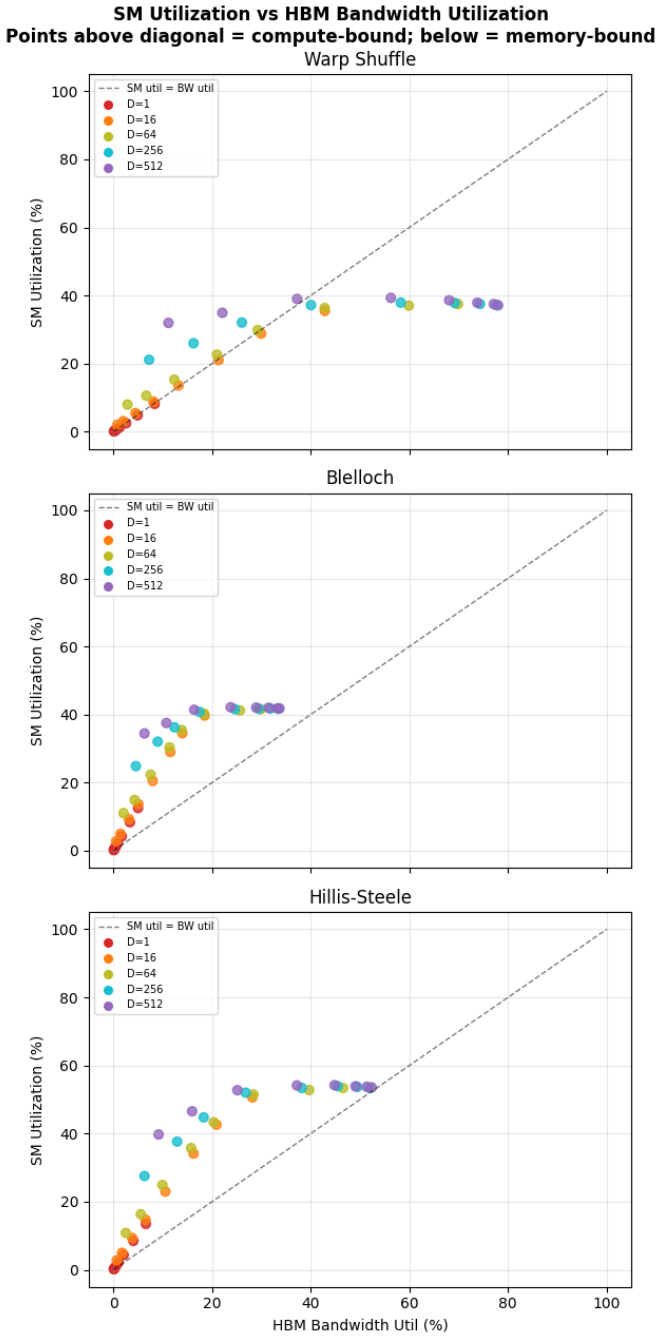


Fig. 5. SM utilization vs. HBM bandwidth utilization across $(L, D)$, measured via Nsight Compute. Warp-shuffle drifts below the diagonal (HBM > SM); Hillis-Steele sits above it (SM > HBM); Blelloch clusters near the diagonal with both metrics lower than Hillis-Steele, consistent with synchronization barriers suppressing both SM and HBM activity simultaneously.

measured on the roofline (Figure 3) falls with $L$ as larger working sets transition from cache-resident to full HBM streaming. The D dimension shifts kernels upward on the roofline (higher GFLOP/s from greater parallelism) but does not meaningfully change arithmetic intensity, since FLOPs and bytes scale proportionally with $D$. All kernels remain firmly memory-bound at all tested $(L, D)$, clustering well left of the 9.6 FLOP/byte ridge point.

**Hillis-Steele degrading at large $L$.** Not confirmed. Hillis-Steele consistently *outperforms* Blelloch at all tested sequence lengths and hidden dimensions. Blelloch's synchronization overhead (Section VI-C) and irregular tree-traversal memory access pattern outweigh its $O(L)$ work advantage in this memory-bound regime. Because the scan is memory-bound, the SM spends most of its time waiting on HBM; Hillis-Steele's extra work fills cycles that would otherwise be idle, making the $O(L \log L)$ vs $O(L)$ distinction effectively irrelevant. No crossover has been observed within our $L$ range.

**Warp-shuffle degrading at large $D$.** This has not been confirmed within our $D$ range. Warp-shuffle maintains a clear performance advantage at all $D$ values. This is explained by the scalar design: each thread holds only $(v_a, v_b)$ in registers regardless of $D$, so register pressure does not increase with $D$. Under the naïve struct-based design, this hypothesis might have been confirmed ($D = 512$ would require 1024 register floats per thread, causing spill), but the scalar refactor eliminated this bottleneck.

### B. Occupancy Tradeoffs

The chunked hierarchical design uses $CHUNK_SIZE = 1024$ threads per block (the hardware limit) maximizing $L$-parallelism within each block. As shown in Figure 4, occupancy rises from ∼48–50% at $D = 1$ to 85–93% at $D = 512$, confirming that the dimension-as-grid design provides a concrete occupancy benefit: more blocks in flight give the SM scheduler a larger pool of warps to switch to while others stall on memory.

Notably, high occupancy does not translate uniformly to high throughput for Blelloch. Despite achieving the highest nominal occupancy among the three kernels (lower shared memory per block means more blocks fit per SM), Blelloch achieves the lowest throughput. This apparent paradox is explained by the SM utilization data in Figure 5: Blelloch's synchronization barriers (Section VI-C) stall all 32 warps simultaneously at each level, consuming SM cycles without issuing memory transactions. High occupancy cannot hide synchronization stalls the way it hides memory latency, because all resident warps are stalled at the same barrier rather than being interleaved.

### C. Scalability Limits and Amdahl's Law

The chunked hierarchical scan has two sources of serial overhead: Phase 2 (scanning the block totals array) and Phase 3 (propagating prefixes back into each chunk). Together these touch $O(num_chunks)$ elements, while Phase 1 processes all $L \times D$ elements in parallel. As $L$ grows, Phase 1 dominates and the serial fraction shrinks: at $L = 1024$ only one chunk exists, so Phases 2 and 3 are trivial; at $L = 131072$ there are 128 chunks, and Phase 1 accounts for $> 99\%$ of the work. By Amdahl's Law, the speedup ceiling rises with $L$ because the parallelizable fraction approaches unity. This is consistent with the data in Table III, where speedup grows

steadily from $L = 1$K to $L = 128$K and has not yet plateaued at our largest tested length.

The practical scalability limit is memory bandwidth, not parallelism. All kernels remain memory-bound across the entire tested parameter space (Section VI-E), with peak throughput reaching only 23% of the A100's 2 TB/s HBM2e ceiling. Further speedup therefore requires either reducing memory traffic (e.g. through kernel fusion with surrounding operations) or moving to higher-bandwidth hardware.

### D. Key Design Takeaway

The most important engineering insight from this project is the **decoupling of $D$ from shared memory** via the dimension-as-grid design. A naïve implementation stores all $D$ scalars per sequence position in shared memory, coupling `CHUNK_SIZE` to $D$: as $D$ grew from 16 to 512, `CHUNK_SIZE` shrank from 1024 to as low as 16, collapsing $L$-parallelism by up to $64\times$ at the operating point where throughput was most critical.

The fix follows directly from the structure of the problem. Mamba's diagonal $\bar{A}$ makes the $D$ dimensions fully independent (Section III-C), and data independence maps naturally to grid independence in CUDA: each $d$-lane becomes a separate thread block, removing $D$ from the shared memory budget entirely. No correctness or generality is lost.

This principle extends beyond SSMs. Any GPU kernel in which one data dimension is independent and currently packed into the inner loop is a candidate for the same refactoring.

## VIII. Conclusion

We implemented and evaluated three parallel prefix scan algorithms — Hillis-Steele, Blelloch, and warp-shuffle — and a chunked hierarchical wrapper that extends each to arbitrary sequence lengths, as the core computational primitive for State Space Model inference on an NVIDIA A100 GPU. All implementations achieve 100% numerical correctness across $L \in \{1\text{K}, 2\text{K}, \ldots, 128\text{K}\}$ and $D \in \{1, 16, 64, 256, 512\}$.

The warp-shuffle variant consistently achieves the highest throughput, reaching $\sim$470 GB/s and $44\times$ speedup over a sequential CPU baseline at $L = 131072$, $D = 512$. The key enabler is the scalar, dimension-as-grid design that pins *CHUNK_SIZE* at 1024 regardless of $D$, eliminates register pressure growth at large $D$, and ensures coalesced HBM access. Contrary to our hypothesis, no register-pressure degradation is observed for warp-shuffle within our $D$ range; contrary to expectation, Hillis-Steele outperforms Blelloch at $D \geq 64$, revealing that synchronization overhead and irregular memory access in the tree-based algorithm outweigh its $O(L)$ work advantage in that regime. All kernels remain memory-bound across the tested parameter space.

**Limitations.** Our evaluation has several constraints. All benchmarks use FP32 arithmetic; FP16 or BF16 would halve memory traffic and could shift the bottleneck balance. Inputs are synthetic ($a_t \sim \text{Uniform}(0.5, 0.99)$, $b_t \sim \mathcal{N}(0, 1)$) rather than drawn from a trained Mamba model. Since the scan has no data-dependent branching, timing results are insensitive to input distribution, but numerical stability under real weight distributions is untested. The scan kernel is profiled in isolation, whereas production Mamba fuses the scan with surrounding linear projections and discretization steps; fusion would reduce HBM round-trips and likely change the performance ranking. Finally, all experiments run on a single A100 GPU with batch size $B = 1$, leaving multi-GPU scaling and batched inference unexplored.

**Future work:** Future directions include conflict-free addressing for Blelloch, batched inputs ($B > 1$), and kernel fusion with surrounding linear projections in a full Mamba layer.

## References


[1] W. D. Hillis and G. L. S. Jr., "Data parallel algorithms," *Communications of the ACM*, vol. 29, no. 12, pp. 1170–1183, 1986.

[2] G. E. Blelloch, "Prefix sums and their applications," Carnegie Mellon University, Tech. Rep. CMU-CS-90-190, 1990.

[3] M. Harris, S. Sengupta, and J. D. Owens, "Parallel prefix sum (Scan) with CUDA," in *GPU Gems 3*. NVIDIA, 2007, ch. 39.

[4] A. Gu, K. Goel, and C. Ré, "Efficiently modeling long sequences with structured state spaces," *arXiv preprint arXiv:2111.00396*, 2022.

[5] J. T. Smith, A. Warrington, and G. W. Whaley, "Simplified state space layers for sequence modeling," *arXiv preprint arXiv:2208.04933*, 2023.

[6] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," *arXiv preprint arXiv:2312.00752*, 2023.


## Appendix

Figure 6 shows per-kernel timing curves for $D \in \{1, 16, 64, 256\}$. As $D$ grows, timing curves diverge, with Hillis-Steele pulling ahead of Blelloch by $D = 64$ and maintaining its advantage through $D = 256$. The pattern at $D = 512$ (Figure 2) represents the clearest expression of these trends.

Figure 7 shows the roofline model for $D \in \{1, 16, 64, 256\}$. All kernels remain memory-bound across all $D$ values, consistent with $D = 512$ (Figure 3). At $D = 1$, points scatter vertically at similar arithmetic intensity, reflecting GPU launch overhead dominating at small working sets. As $D$ grows from 16 to 256, the cluster tightens and shifts toward the bandwidth ceiling as the working set grows large enough to saturate HBM. Blelloch's separation from warp-shuffle and Hillis-Steele, visible at $D = 512$, begins to emerge at $D = 64$ and is clear by $D = 256$, consistent with synchronization overhead becoming dominant as block count grows with $D$.

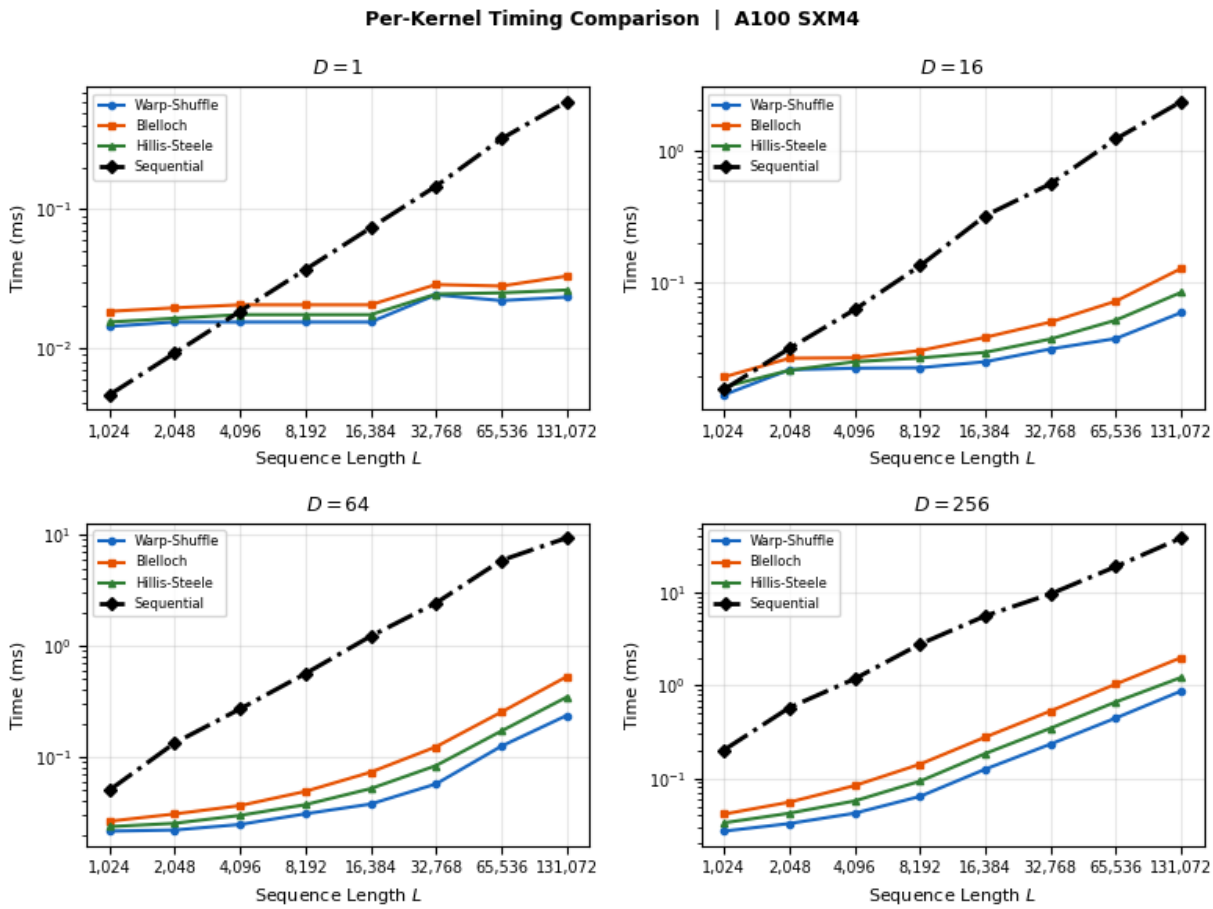


Fig. 6. Per-kernel wall-clock time vs. sequence length $L$ for $D \in \{1, 16, 64, 256\}$. Sequential baseline shown as dash-dot black. Both axes log-scale. A100 SXM4-40GB.

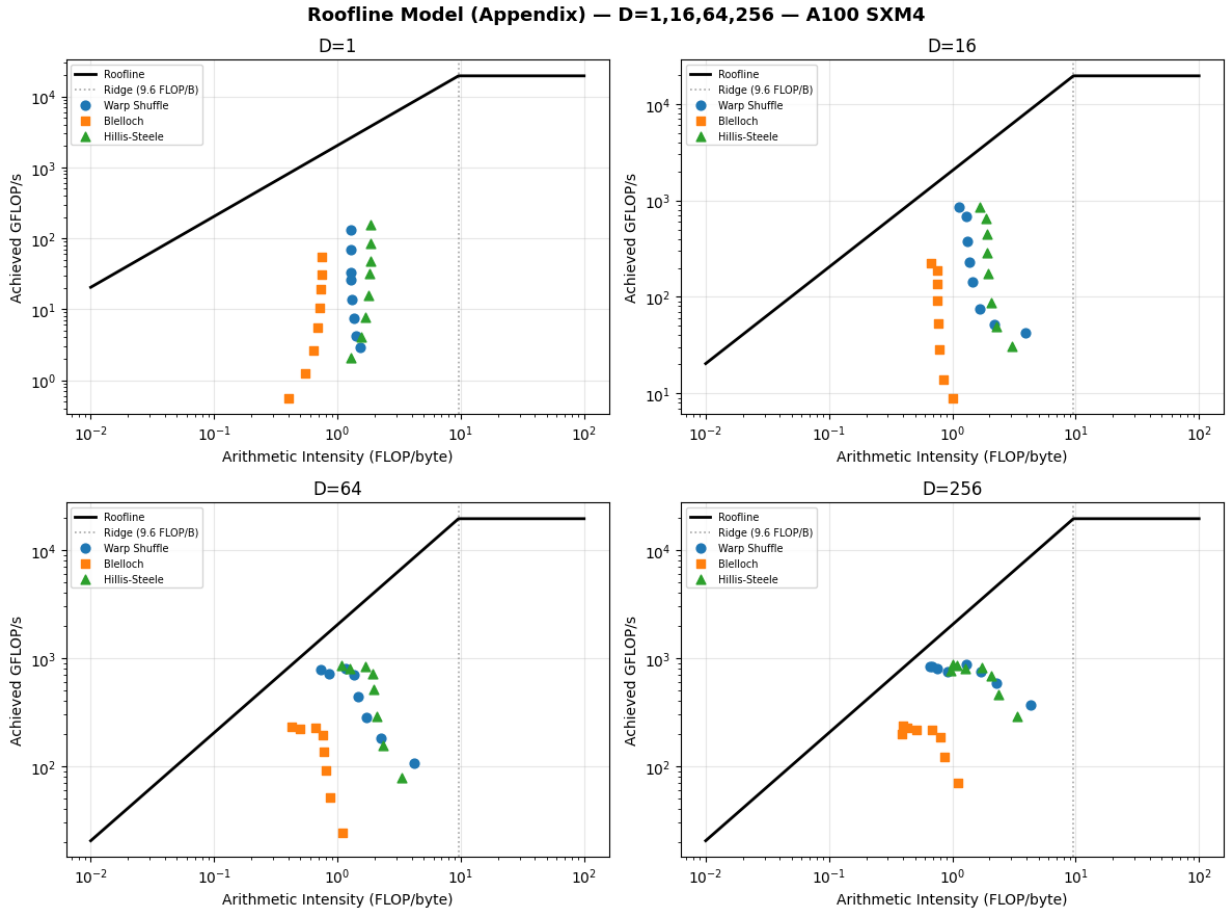


Fig. 7. Roofline model for $D \in \{1, 16, 64, 256\}$. All kernels are memory-bound at all $D$ values. Blelloch's separation below warp-shuffle and Hillis-Steele grows with $D$, consistent with synchronization overhead increasing as working set size and block count grow.